PYTHON PROGRAMMING
43
```
j=1
while i<=3:
print("",end=" ")

while j<=1:
print ("CSE DEPT",end="")
j=j+1
i=i+1
print()
```
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/wh3.py
CSE DEPT

4. ----------

```
i = 1
while (i < 10):
print (i)
i = i+1
```
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/wh4.py
1
2
3
4
5
6
7
8
9

2. ----------
```
a = 1
b = 1
while (a<10):
print ('Iteration',a)
a = a + 1
b = b + 1
```
PYTHON PROGRAMMING
44
```
if (b == 4):
break
print ('While loop terminated')
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/wh5.py
Iteration 1
Iteration 2
Iteration 3
While loop terminated

```
count = 0
while (count < 9):
```

```
print("The count is:", count)
count = count + 1
print("Good bye!")
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/wh.py =
The count is: 0
The count is: 1
The count is: 2
The count is: 3
The count is: 4
The count is: 5
The count is: 6
The count is: 7
The count is: 8
Good bye!

For loop :
Python for loop is used for repeated execution of a group of statements for the desired number of times. It iterates over the items of lists, tuples, strings, the dictionaries and other iterable objects
Syntax: for var in sequence:
Statement(s) A sequence of values assigned to var in each iteration
Holds the value of item
in sequence in each iteration

Sample Program :

```
numbers = [1, 2, 4, 6, 11, 20]
seq=0
for val in numbers:
seq=val*val
print(seq)
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/fr.py
1
4
16
36
121
400

Flowchart:

Iterating over a list:

```
#list of items
list = ['M','R','C','E','T']
i = 1

#Iterating over the list
for item in list:
print ('college ',i,' is ',item)
i = i+1
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/lis.py
college 1 is M
college 2 is R
college 3 is C
college 4 is E
college 5 is T

Iterating over a Tuple:
tuple = (2,3,5,7)
print ('These are the first four prime numbers ')
#Iterating over the tuple
for a in tuple:
print (a)

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/fr3.py
These are the first four prime numbers
2
3
5
7

Iterating over a dictionary:
#creating a dictionary
college = {"ces":"block1","it":"block2","ece":"block3"}

#Iterating over the dictionary to print keys
print ('Keys are:')
PYTHON PROGRAMMING
47
for keys in college:
print (key s)

#Iterating over the dictionary to print values
print ('Values are:')
for blocks in college.values():
print(blocks)

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/dic.py
Keys are:
ces
it
ece
Values are:
block1
block2
block3

Iterating over a String :
#declare a string to iterate over
college = ''

#Iterating over the string

```
for name in college:
print (name)
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/strr.py
M
R
C
E
T

Nested For loop:
When one Loop defined within another Loop is called Nested Loops.
Syntax:
for val in sequence:
for val in sequence:

statements
statements

```
# Example 1 of Nested For Loops (Pattern Programs)
for i in range(1,6):
for j in range(0,i):
print(i, end=" ")
print('')
```
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/nesforr.py
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5

```
# Example 2 of Nested For Loops (Pattern Programs)
for i in range(1,6):
for j in range(5,i -1,-1):
print(i, end=" ")
print('')
```
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/nesforr.py
Output:
1 1 1 1 1
2 2 2 2
3 3 3
4 4

Break and continue:
In Python, break and continue statements can alter the flow of a normal loop. Sometimes we wish to terminate the current iteration or even the whole loop without checking test expression. The break and continue statements are used in these cases.

Break:
The break statement term inates the loop containing it and c ontrol of the progra m flows to the statement immediately after the body of the loop. If break statement is inside a nested loop (loop inside another loop), break will terminate the innermost loop.

Flowchart:

The following shows the working of break statement in for and while loop :
for var in sequence:
# code inside for loop
If condition:
break (if break condition satisfies it jumps to outside loop)
code inside for loop
code outside for loop

PYTHON PROGRAMMING
50
while test expression
code inside while loop
If condition:
break (if break condition satisfies it jumps to outside loop)
code inside while loop
code outside while loop

Example:
for val in " COLLEGE":
if val == " ":
break
print(val)

print("The end")

Output:
M
R
C
E
T
The end

Program to disp lay all the elements before number 88

for num in [11, 9, 88, 10, 90, 3, 19]:
print(num)
if(num==88):
print("The number 88 is found")
print("Terminating the loop")
break

Output:
11
9
88
The number 88 is found
PYTHON PROGRAMMING
51
Terminating the loop

#---------------------------------------
for letter in "Python": # First Example
if letter == "h":

```
break
print("Current Letter :", letter )
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/br.py =
Current Letter : P
Current Letter : y
Current Letter : t
Continue:
The continue statement is used to skip the rest of the code inside a loop for the current iteration only. Loop does not terminate but continues on with the next iteration.
Flowchart:

The following shows the working of break statement in for and while loop :


```
for var in sequence:
# code inside for loop
If condition:
continue (if break condition satisfies it jumps to outside loop)
# code inside for loop
# code outside for loop

while test ex pression
# code inside while loop
If condition:
continue(if break condition satisfies it jumps to outside loop)
# code inside while loop
# code outside while loop
```

Example:

```
# Program to show the use of continue statement inside loops

for val in "string":
if val == "i":
continue
print(val)

print("The end")
```

Output:

C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/cont.py
s
t
r
n
g
The end

```
# program to display only odd numbers

for num in [20, 11, 9, 66, 4, 89, 44]:
```



```
# Skipping the iteration when number is even
```

```
if num%2 == 0:
continue
# This statement will be skipped for all even numbers
print(num)
```

Output:

```
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/cont2.py
11
9
89
```

```
#------------------
for letter in "Python": # First Example
if letter == "h":
continue
print("Current Letter :", letter)
```

Output:

```
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/con1.py
Current Letter : P
Current Letter : y
Current Letter : t
Current Letter : o
Current Letter : n
```

Pass:

In Python programming, pass is a null statement. The difference between
a comment and pass statement in Python is that, while the interpreter ignores a comment entirely, pass is not ignored.
pass is just a placeholder for functionality to be added later.

Example:

```
sequence = {'p', 'a', 's', 's'}
for val in sequence:
pass
```

Output:

```
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/f1.y.py
PYTHON PROGRAMMING
54
>>>
```

Similarily we can also write,

```
def f(arg): pass # a function that does nothing (yet)

class C: pass # a class with no methods (yet)
```


UNIT – III
FUNCTIONS, ARRAYS
Fruitful functions: return values, parameters, local and global scope, function composition, recursion; Strings: string slices, immutability, string functions and methods, string module;

Python arrays, Access the Elements of an Array, ar ray methods.
Functions, Arrays :
Fruitful functions:
We write functions that return values, which we will call fruitful functions . We have seen the return statement before, but in a fruitful function the return statement includes a return value . This statement means: "Return immediately from this function and use the following expression as a return value."
(or)
Any function that returns a value is called Fruitful function. A Function that does not return a value is called a void function
Return values:
The Keyword return is used to return back the value to the called function.

```
# returns the area of a circle with the given radius:
def area(radius):
temp = 3.14 * radius**2
return temp
print(area(4))
```

(or)

```
def area(radius):
return 3.14 * radius**2
print(area(2))
```

Sometimes it is useful to have multiple return statements, one in each branch of a conditional:

```
def absolute_value(x):
if x < 0:
```



```
return -x
else:
return x
```

Since these return statements are in an alternative conditional, only one will be executed. As soon as a return statement executes, the function terminates without executing any subsequent statements. Code that appears after a return statement, or any other place the flow of execution can never reach, is called dead code.
In a fruitful function, it is a good idea to ensure that every possible path through the program hits a return statement. For example:

```
def absolute_value(x):
if x < 0:
return -x
if x > 0:
return x
```

This fu nction is incorrect because if x happens to be 0, both conditions is true, and the function ends without hitting a return statement. If the flow of execution gets to the end of a function, the return value is None, which is not the absolute value of 0.

```
>>> print absolute_value(0)
None
```

By the way, Python provides a built -in function called abs that computes absolute values.

```
# Write a Python function that takes two lists and returns True if they have at least one
common member .
def common_data(list1, list2):
for x in list1:
for y in list2:
if x == y:
```

```
result = True
return result
print(common_data([1,2,3,4,5], [1,2,3,4,5]))
print(common_data([1,2,3,4,5], [1,7,8,9,510]))
print(common_data([1,2,3,4,5], [6,7,8 ,9,10]))
```

Output:
C:\Users \ \AppData \Local \Programs \Python \Python38 -32\pyyy \fu1.py
PYTHON PROGRAMMING
57
True
True
None

```
#-----------------
def area(radius):
b = 3.14159 * radius**2
return b
```

Parameters:
Parameters are passed during the definition of function while Arguments are passed during the function call .

Example:

```
#here a and b are parameters

def add(a,b): #//function definition
return a+b

#12 and 13 are arguments
#function call
result=add(12,13)
print(result)
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/paraarg.py
25

Some examples on functions:

```
# To display vandemataram by using function use no args no return type
#function defination
def display():
print("vandemataram")
print("i am in mai n")
```



```
#function call
display()
print("i am in main")
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/fu1.py
i am in main
vandemataram
i am in main

```
#Type1 : No parameters and no return type
def Fun1() :
print("function 1")
Fun1()
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/fu1.py
function 1

```
#Type 2: with param with out return type
def fun2(a) :
print(a)
fun2("hello")
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/fu1.py
Hello

```
#Type 3 : without param with return type
def fun3():
return "welcome to python"
print(fun3())
```


Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/fu1.py
welcome to python

```
#Type 4: with param with return type
def fun4(a):
return a
print(fun4("python is better then c"))
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/fu1.py
python is better then c

Local and Global scope:
Local Scope:
A variable which is defined inside a function is local to that function. It is accessible from the point at which it is defined until the end of the function, and exists for as long as the function is executing
Global Scope:
A variable which is defined in the main body of a file is called a global variable. It will be visible thro ughout the file, and also inside any file which imports that file.
■ The variable defined inside a function can also be made global by using the global statement.

```
def function_name(args):
.....
global x #declaring global variable inside a function
..... ..
# create a global variable
```



```
x = "global"

def f():
```

```
print("x inside :", x)

f()
print("x outside:", x)
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/fu1.py
x inside : global
x outside: global

```
# create a local variable
def f1 ():
y = "local"
print(y)
f1()
```

Output:
local

■ If we try to access the local variable outside the scope for example,

```
def f2 ():
y = "local"

f2()
print(y)
```

Then when we try to run it shows an error,

Traceback (most recent call last):
File "C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/fu1.py", line
6, in
PYTHON PROGRAMMING
61
print(y)
NameError: name 'y' is not defined

The output shows an error, b ecause we are trying to access a local variable y in a global scope whereas the local variable only works inside f2() or local scope.

```
# use local and global variables in same code
x = "global"

def f3():
global x
y = "local"
x = x * 2
print(x)
print(y)

f3()
```

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/fu1.py
globalglobal
local

■ In the above code, we declare x as a global and y as a local variable in the f3(). Then, we use multiplication operator * to modify the global variable x and we print both x and y.
■ After calling the f3(), the value of x becomes global global because we used the x * 2 to print two times global. After that, we print the value of local variable y i.e local.

```
# use Global variable a nd Local variable with same name
x = 5

def f4():
x = 10
print("local x:", x)

f4()
print("global x:", x)
```


Output:

```
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/fu1.py
local x: 10
global x: 5
```

Function Composition:
Having two (or more) functions where the output of one function is the input for another. So for example if you have two functions FunctionA and FunctionB you compose them by doing the following.

```
FunctionB(FunctionA(x))
```

Here x is the input for FunctionA and the result of that is the input for FunctionB.
Example 1:

```
#create a function compose2
>>> def compose2(f, g):
return lambda x:f(g(x))

>>> def d(x):
return x*2

>>> def e(x):
return x+1

>>> a=compose2(d,e) # FunctionC = compose(FunctionB,FunctionA)
>>> a(5) # FunctionC(x)
12
```

In the above program we tried to compose n functions with the main function created.
Example 2:

```
>>> colors=('red','green','blue')
>>> fruits=['orange','banana','cherry']
>>> zip(colors,fruits)
```



```
>>> list(zip(colors,fruits))
[('red', 'orange'), ('green', 'banana'), ('blue', 'cherry')]
```

Recursion:
Recursion is the process of defining something in terms of itself.
Python Recursive Function
We know that in Python, a function can call other functions. It is even possible for the function to call itself. These type of construct are termed as recursive functions.
Factorial of a numbe r is the product of all the integers from 1 to that number. For example, the factorial of 6 (denoted as 6!) is 1*2*3*4*5*6 = 720.
Following is an example of recursive function to find the factorial of an integer.

```
# Write a program to factorial using recurs ion
def fact(x):
if x==0:
result = 1
else :
result = x * fact(x -1)
return result
print("zero factorial",fact(0))
print("five factorial",fact(5))
```

Output:

```
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/rec.py
zero factorial 1
five factorial 120
```

```
def calc_factorial(x):
"""This is a recursive function
to find the factorial of an integer"""

if x == 1:
return 1
else:
return (x * calc_factorial(x -1))
```

PYTHON PROGRAMMING
64

```
num = 4
print( "The factorial of", num, "is", calc_factorial(num))
```

Output:

```
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/rec.py
The factorial of 4 is 24
```

Strings:
A string is a group / a sequence of characters. Since Python has no provision for arrays, we simply use strings. This is how we declare a string. We can use a pair of single or double quotes. Every string object is of the type ‘str’.

```
>>> type("name")

>>> name=str()
>>> name
''
>>> a=str('')
>>> a
''
>>> a=str()
```

```
>>> a[2]
'c'
>>> fruit = 'banana'
>>> letter = fruit[1]
```
The second statement selects character number 1 from fruit and assigns it to letter. The expression in brackets is called an index. The index indicates which character in the sequence we want

String slices:
A segment of a string is called a slice. Selecting a slice is similar to selecting a character:

Subsets of strings can be taken using the slice operator ([ ] and [:]) with indexes starting at 0 in the beginning of the string and working their way from -1 at the end.
Slice out substrings, sub lists, sub Tuples using index.

Syntax: [Start: stop: steps]

■ Slicing will start from index and will go up to stop in step of steps.
■ Default value of start is 0,

■ Stop is last index of list
■ And for step default is 1

For example 1–
```
str = 'Hello World!'
print str # Prints complete string
print str[0] # Prints first character of the string
print str[2:5] # Prints characters starting from 3rd to 5th
print str[2:] # Prints string starting from 3rd character print
str * 2 # Prints string two times
print str + "TEST" # Prints concatenated string
```
Output:
```
Hello World!
H
llo
llo World!
Hello World!Hello World!
Hello World!TEST
```
Example 2:
```
>>> x='computer'
>>> x[1:4]
'omp'
>>> x[1:6:2]
'opt'
>>> x[3:]
```

```
'puter'
>>> x[:5]
'compu'
>>> x[ -1]
'r'
>>> x[ -3:]
'ter'
>>> x[: -2]
```

'comput'
>>> x[:: -2]
'rtpo'
>>> x[:: -1]
'retupmoc'

Immutability:
It is tempting to use the [] operator on the left side of an assignment, with the intention of changing a character in a string.
For example:

>>> greeting=' college!'
>>> greeting[0]='n'

TypeError: 'str' object does not support item assignment

The reason for the error is that strings are immutable , which means we can't change an existing string. The best we can do is creating a new string that is a variation on the original:

>>> greeting = 'Hello, world!'
>>> new_greeting = 'J' + greeting[1:]
>>> new_greeting
'Jello, world!'

Note: The plus (+) sign is the string concatenation operator and the asterisk (*) is the repetition operator

String functions and methods:

There are many methods to operate on String.

S.no Method name Description
1. isalnum() Returns true if string has at least 1 character and all characters are alphanumeric and false otherwise.
2. isalpha() Returns true if string has at least 1 character and all characters are alphabetic and false otherwise.
3. isdigit() Returns true if string contains only digits and false otherwise.
4. islower() Returns true if string has at least 1 cased character and all cased characters are in lowercase and false otherwise.
5. isnumeric() Returns true if a string contains only numeric characters and false otherwise.
6. isspace() Returns true if string contains only whitespace characters and false otherwise.
7. istitle() Returns true if string is properly "titlecased" and false otherwise.
8. isupper() Returns true if string has at least one cased character and all cased characters are in uppercase and false otherwise.
9. replace(old, new [, max]) Replaces all occurrences of old in string with new or at most max occurrences if max given.
10. split() Splits string according to delimiter str (space if not

provided) and returns list of substrings;
11. count() Occurrence of a string in another string
12. find() Finding the index of the first occurrence of a string
in another string
13. swapcase() Converts lowercase letters in a string to uppercase
and viceversa
14. startswith(str,
beg=0,end=le
n(string)) Determines if string or a substring of string (if starting index
beg and ending index end are given) starts with substring str;
returns true if so and false
otherwise.

Note:
All the string methods will be returning either true or false as the result

1. isalnum():
PYTHON PROGRAMMING
68
Isalnum() method returns true if string has at least 1 character and all characters are
alphanumeric and false otherwise.

Syntax:
String.isalnum()

Example:
>>> string="123alpha"
>>> string.isalnum() True

2. isalpha():
isalpha() method returns true if string has at least 1 character and all characters are
alphabetic and false otherwise.

Syntax:
String.isalpha()

Example:
>>> string="nikhil"
>>> string.isalpha()
True

3. isdigit():
isdigit() returns true if string contains only digits and false otherwise.

Syntax:
String.isdigit()

Example:
>>> string="123456789"
>>> string.isdigit()
True

4. islower():
Islower() returns true if string has characters that are in lower case and false otherwise.

Syntax:

String.islower()

Example:
```
>>> string="nikhil"
>>> string.islower()
True
```

5. isnumeric():
isnumeric() method returns true if a string contains only numeric characters and false otherwise.

Syntax:
String.isnumeric()

Example:
```
>>> string="123456789"
>>> string.isnumeric()
True
```

6. isspace():
isspace() returns true if string contains only whitespace characters and false otherwise.

Syntax:
String.isspace()

Example:
```
>>> string=" "
>>> string.isspace()
True
```

7. istitle()
istitle() method returns true if string is properly “titlecased”(starting letter of each word is capital) and false otherwise

Syntax:
String.istitle()

Example:
```
>>> string="Nikhil Is Learning"
>>> string.istitle()
True
```

8. isupper()
isupper() returns true if string has characters that are in uppercase and false otherwise.

Syntax:
String.isupper()

Example:
```
>>> string="HELLO"
>>> string.isupper()
True
```

9. replace()

replace() method replaces all occurrences of old in string with new or at most max occurrences if max given.

Syntax:
String.replace()

Example:
>>> string="Nikhil Is Learning"
>>> string.replace('Nikhil','Neha')
'Neha Is Learning'

10. split()
split() method splits the string according to delimiter str (space if not provided)

Syntax:
String.split()

Example:
>>> string="Nikhil Is Learning"
>>> string.split()
PYTHON PROGRAMMING
71
['Nikhil', 'Is', 'Learning']

11. count()
count() method counts the occurrence of a string in another string Syntax:
String.count()

Example:
>>> string='Nikhil Is Learning'
>>> string.count('i')
3

12. find()
Find() method is used for finding the index of the first occurrence of a string in another string

Syntax:
String.find(„string■)

Example:
>>> string="Nikhil Is Learning"
>>> string.find('k')
2
13. swapcase()
converts lowercase letters in a string to uppercase and viceversa

Syntax:
String.find(„string■)

Example:
>>> string="HELLO"
>>> string.swapcase()
'hello'

14. startswith()

Determines if string or a substring of string (if starting index beg and ending index end are given) starts with substring str; returns true if so and false otherwise.
PYTHON PROGRAMMING
72
Syntax:
String.startswith(„string■)

Example:
>>> string="Nikhil Is Learning"
>>> string.startswith('N')
True

15. endswith()
Determines if string or a substring of string (if starting index beg and ending index end are given) ends with substring str; returns true if so and false otherwise.

Syntax:
String.endswith(„string■)

Example:
>>> string="Nikhil Is Learning"
>>> string.startswith('g')
True

String module:

This module contains a number of functions to process standard Python strings. In recent versions, most functions are available as string methods as well.

It’s a built -in module and we have to import it before using any of its constants and classes

Syntax: import string

Note:
help(string) --- gives the information abo ut all the variables ,functions, attributes and classes to be used in string module.

Example:
import string
print(string.ascii_letters)
print(string.ascii_lowercase)
print(string.ascii_uppercase)
print(string.digits)
PYTHON PROGRAMMING
73
print(string.hexdigits)
#print(string.whitespace)
print(string.punctuation)

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/strrmodl.py
==========
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz
ABCDEFGHIJKLMNOPQRSTUVWXYZ
0123456789

0123456789abcdefABCDEF
!"#$%&'()*+, -./:;<=>?@[\]^_`{|}~

Python String Module Classes
Python string module contains two classes – Formatter and Template.
Formatter
It behaves exactly same as str.format() function. This class becomes useful if you want to subclass it and define your own format string syntax.
Syntax: from string import Formatter
Template
This class is used to create a string template for simpler string substitutions
Syntax: from string import Template
Python arrays:
Array is a container which can hold a fix number of items and these items should be of the same type. Most of the data structures make use of arrays to implement their algorithms. Following are the important terms to understand the concept of Array.
■ Element – Each item stored in an array is called an element.
■ Index – Each location of an element in an array has a numerical index, which is used to identify the element.
Array Representation
PYTHON PROGRAMMING
74
Arrays can be declared in various ways in different languages. Below is an illustration.
Elements
Int array [10] = {10, 20, 30, 40, 50, 60, 70, 80, 85, 90}

Type Name Size Index 0

As per the above illustration, following are the important points to be considered.
■ Index starts with 0.
■ Array length is 10 which means it can store 10 elements.
■ Each element can be accessed via its index. For example, we can fetch an element at index 6 as 70
Basic Operations
Following are the basic operations supported by an array.
■ Traverse – print all the array elements one by one.
■ Insertion – Adds an element at the given index.
■ Deletion – Deletes an element at the given index.
■ Search – Searches an element using the given index or by the value.
■ Update – Updates an element at the given index.
Array is created in Python by importing array module to the python program. Then the array is declared as shown below.
from array import *
arrayName=array(typecode, [initializers])
Typecode are the codes that are used to define the type of value the array will hold. Some common typecodes used are:

Typecode Value
b Represents signed integer of size 1 byte/td>
B Represents unsigned integer of size 1 byte
PYTHON PROGRAMMING
75
c Represents character of size 1 byte
i Represents signed integer of size 2 bytes
I Represents unsigned integer of size 2 bytes
f Represents floating point of size 4 bytes

d Represents floating point of size 8 bytes

Creating an array:

```
from array import *
array1 = array('i', [10,20,30,40,50])
for x in array1:
print(x)
```

Output:

```
>>>
RESTART: C:/Users//AppData/Local/Progra ms/Python/Python38 -32/arr.py
10
20
30
40
50
```

Access the element s of an A rray:

Accessing Array Element

We can access each element of an array using the index of the element.

```
from array import *
array1 = array('i', [10,20,30,40,50])
print (array1[0])
print (array1[2])
```


Output:

```
RESTART: C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/arr2.py
10
30
```

Array methods:

Python has a set of built -in methods that you can use on lists/arrays.

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the first item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

Note: Python does not have built -in support for Arrays, but Python Lists can be used instead.

Example:

```
>>> college=["","it","cse"]
>>> college.append("autonomous")
>>> college
['', 'it', 'cse', 'autonomous']
>>> college.append("eee")
>>> college.append("ece")
```

```
>>> college
['', 'it', 'cse', 'autonomous', 'eee', 'ece']
>>> college.pop()
'ece'
>>> college
['', 'it', 'cse', 'autonomous', 'eee']
>>> college.pop(4)
'eee'
>>> college
['', 'it', 'cse', 'autonomous']
>>> college.remove("it")
>>> college
['', 'cse', 'autonomous']
```

UNIT – IV
LISTS, TUPLES, DICTIONARIES
Lists: list operations, list slices, list methods, list loop, mutability, aliasing, cloning lists, list parameters, list comprehension; Tuples: tuple assignment, tuple as return value, tuple comprehension; Dictionaries : operations and methods, comprehension;
Lists, Tuples, Dictionaries :
List:
■ It is a general purpose most widely used in data structures
■ List is a collection which is ordered and changeable and allows duplicate members. (Grow and shrink as needed, sequence type, sortable).
■ To use a list, you must declare it first. Do this using square brackets and separate values with commas.
■ We can construct / create list in many ways.
Ex:
```
>>> list1=[1,2,3,'A','B',7,8,[10,11]]
>>> print(list1)
[1, 2, 3, 'A', 'B', 7, 8, [10, 11]]
----------------------
>>> x=list()
>>> x
[]
--------------------------
>>> tuple1=(1,2,3,4)
>>> x=list(tuple1)
>>> x
[1, 2, 3, 4]
```

List operations:
These operations include indexing, slicing, adding, multiplying, and checking for membership
Basic List Operations :
Lists respond to the + and * operators much like strings; they mean concatenation and repetition here too, except that the result is a new list, not a string.

Python Expression	Results	Description
len([1, 2, 3])	3	Length
[1, 2, 3] + [4, 5, 6]	[1, 2, 3, 4, 5, 6]	Concatenation
['Hi!'] * 4	['Hi!', 'Hi!', 'Hi!', 'Hi!']	Repetition
3 in [1, 2, 3]	True	Membership
for x in [1, 2, 3]: print x,	1 2 3	Iteration

Indexing, Slicing, and Matrixes

Because lists are sequences, indexing and slicing work the same way for lists as they do for strings.
Assuming following input –
L = [' ', 'college ', ' !']

Python Expression Results Description
L[2] Offsets start at zero
PYTHON PROGRAMMING
80
L[-2] college Negative: count from the right
L[1:] ['college ', ' !'] Slicing fetches sections

List slices:
```
>>> list1=range(1,6)
>>> list1
range(1, 6)
>>> print(list1)
range(1, 6)
>>> list1=[1,2,3,4,5,6,7,8,9,10]
>>> list1[1:]
[2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> list1[:1]
[1]
>>> list1[2:5]
[3, 4, 5]
>>> list1[:6]
[1, 2, 3, 4, 5, 6]
>>> list1[1:2:4]
[2]
>>> list1[1:8:2]
[2, 4, 6, 8]
```
List methods :
The list data type has some more methods. Here are all of the methods of list objects:
■ Del()
PYTHON PROGRAMMING
81
■ Append()
■ Extend()
■ Insert()
■ Pop()
■ Remove()
■ Reverse()
■ Sort()
Delete: Delete a list or an item from a list
```
>>> x=[5,3,8,6]
>>> del(x[1]) #deletes the index position 1 in a list
>>> x
[5, 8, 6]
------------
>>> del(x)
>>> x # complete list gets deleted
```
Append: Append an item to a list
```
>>> x=[1,5,8,4]
>>> x.append(10)
>>> x
[1, 5, 8, 4, 10]
```

Extend: Append a sequence to a list.

```
>>> x=[1,2,3,4]
>>> y=[3,6,9,1]
>>> x.extend(y)
>>> x
[1, 2, 3, 4, 3, 6, 9, 1]
```

Insert: To add an item at the specified index, use the insert () method:

```
>>> x=[1,2,4,6, 7]
```



```
>>> x.insert(2,10) #insert(index no, item to be inserted)
>>> x
[1, 2, 10, 4, 6, 7]
-------------------------
>>> x.insert(4,['a',11])
>>> x
[1, 2, 10, 4, ['a', 11], 6, 7]
```

Pop: The pop() method removes the specified index, (or the last item if index i s not specified) or simply p ops the last item of list and returns the item .

```
>>> x=[1, 2, 10, 4, 6, 7]
>>> x.pop()
7
>>> x
[1, 2, 10, 4, 6]
-----------------------------------
>>> x=[1, 2, 10, 4, 6]
>>> x.pop(2)
10
>>> x
[1, 2, 4, 6]
```

Remove: The remove() method removes the specified item from a given list.

```
>>> x=[1,33,2,10,4,6]
>>> x.remove(33)
>>> x
```



```
[1, 2, 10, 4, 6]
>>> x.remove(4)
>>> x
[1, 2, 10, 6]
```

Reverse: Reverse the order of a given list.

```
>>> x=[1,2,3,4,5,6,7]
>>> x.reverse()
>>> x
[7, 6, 5, 4, 3, 2, 1]
```

Sort: Sorts the elements in ascending order

```
>>> x=[7, 6, 5, 4, 3, 2, 1]
>>> x.sort()
>>> x
[1, 2, 3, 4, 5, 6, 7]
----------------------
>>> x=[10,1,5,3,8,7]
>>> x.sort()
>>> x
[1, 3, 5, 7, 8, 10]
```

List loop:

Loops are control structures used to repeat a given section of code a certain number of times or until a particular condition is met.
Method #1: For loop
#list of items
list = ['M','R','C','E','T']
PYTHON PROGRAMMING
84
i = 1

#Iterating over the list
for item in list:
print ('college ',i,' is ',item)
i = i+1
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/lis.py
college 1 is M
college 2 is R
college 3 is C
college 4 is E
college 5 is T

Method #2: For loop and range()
In case we want to use the traditional for loop which iterates from number x to number y.
# Python3 code to iterate over a list
list = [1, 3, 5, 7, 9]

# getting length of list
length = len(list)

# Iterating the index
# same as 'for i in range(len(list))'
for i in range(length):
print(list[i])

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/listlooop.py
1
3
5
7
9
Method #3 : using while loop

# Python3 code to iterate over a list
list = [1, 3, 5, 7, 9]

Getting length of list
PYTHON PROGRAMMING
85
length = len(list)
i = 0

# Iterating using while loop
while i < length:
print(list[i])
i += 1

Mutability :
A mutable object can be changed after it is created, and an immutable object can't.
Append: Append an item to a list
```
>>> x=[1,5,8,4]
>>> x.append(10)
>>> x
[1, 5, 8, 4, 10]
```
Extend: Append a sequence to a list.
```
>>> x=[1,2,3,4]
>>> y=[3,6,9,1]
>>> x.extend(y)
>>> x
```
Delete: Delete a list or an item from a list
```
>>> x=[5,3,8,6]
>>> del(x[1]) #deletes the index position 1 in a list
>>> x
[5, 8, 6]
```
Insert: To add an item at the specified index, use the insert () method:
```
>>> x=[1,2,4,6,7]
>>> x.insert(2,10) #insert(index no, item to be inserted)
```

```
>>> x
[1, 2, 10, 4, 6, 7]
-------------------------
>>> x.insert(4,['a',11])
>>> x
[1, 2, 10, 4, ['a', 11], 6, 7]
```
Pop: The pop() method removes the specified index, (or the last item if index is not specified) or simply p ops the last item of list and returns the item .
```
>>> x=[1, 2, 10, 4, 6, 7]
>>> x.pop()
7
>>> x
[1, 2, 10, 4, 6]
-----------------------------------
>>> x=[1, 2, 10, 4, 6]
>>> x.pop(2)
10
>>> x
[1, 2, 4, 6]
```
Remove: The remove() method removes the specified item from a given list.
```
>>> x=[1,33,2,10,4,6]
>>> x.remove(33)
>>> x
[1, 2, 10, 4, 6]
```

```
>>> x.remove(4)
>>> x
[1, 2, 10, 6]
```
Reverse: Reverse the order of a given list.
```
>>> x=[1,2,3,4,5,6,7]
>>> x.reverse()
>>> x
```

[7, 6, 5, 4, 3, 2, 1]
Sort: Sorts the elements in ascending order
>>> x=[7, 6, 5, 4, 3, 2, 1]
>>> x.sort()
>>> x
[1, 2, 3, 4, 5, 6, 7]

>>> x=[10,1,5,3,8,7]
>>> x.sort()
>>> x
[1, 3, 5, 7, 8, 10]

Aliasing:
1. An alias is a second name for a piece of data, often easier (and more useful) than making a copy.
2. If the data is immutable, aliases don't matter because the data can't change.
3. But if data can change, aliases can result in lot of hard – to – find bugs.
4. Aliasing happens whenever one variable's value is assigned to another variable.

For ex:
a = [81, 82, 83]
PYTHON PROGRAMMING
88
b = [81, 82, 83]
print(a == b)
print(a is b)
b = a
print(a == b)
print(a is b)
b[0] = 5
print(a)
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/alia.py
True
False
True
True
[5, 82, 83]

Because the same list has two different names, a and b, we say that it is aliased . Changes made with one alias affect the other. In the example above , you can see that a and b refer to the same list after executing the assignment statement b = a.
Cloning Lists:
If we want to modify a list and also keep a copy of the original, we need t o be able to make a copy of the list itself, not just the reference. This process is sometimes called cloning , to avoid the ambiguity of the word copy.
The easiest way to clone a list is to use the slice operator. Taking any slice of a creates a new list. In this case the slice happens to consist of the whole list.
Example:
a = [81, 82, 83]
b = a[:] # make a clone using slice
print(a == b)
print(a is b)
b[0] = 5
print(a)
print(b)

Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/pyyy/clo.py
True
False
[81, 82, 83]
[5, 82, 83]
Now we are free to make changes to b without worrying about a

List parameters:
Passing a list as an argument actually passes a reference to the list, not a copy of the list. Since lists are mutable, changes made to the elements referenced by the parameter change the same list that the argument is referencing.
# for example, the function below takes a list as an argument and multiplies each element in the list by 2:
def doubleStuff(List):
""" Overwrite each element in aList with double its value. """
for position in range(len(List)):
List[position] = 2 * List[position]

things = [2, 5, 9]
print(things)
doubleStuff(things)
print(things)
Output:
C:/Users//AppData/Local/Programs/Python/Python38 -32/lipar.py ==
[2, 5, 9]
[4, 10, 18]